

# DOCUMENTO TÉCNICO — SPRINT 3

## Sistema de Gerenciamento de Estação de Recarga (ChargeGrid Intelligence)

**Instituição:** FIAP – Faculdade de Informática e Administração Paulista

**Curso:** Ciência da Computação

**Matéria:** Data Structures and Algorithms

**Professor:** Erick Toshio Yamamoto

**Linguagem:** C (GCC)

**Grupo:** Grupo 03

## 1. EXPLICAÇÃO DA ESTRUTURA CRIADA (STRUCT E TYPEDEF)

---

Para armazenar e manipular os dados de várias sessões de recarga no mesmo programa, foi criada uma estrutura em C utilizando `struct` e `typedef` chamada `Sessao`. Essa estrutura junta diferentes tipos de dados necessários para representar uma recarga em um único bloco de memória:

```
typedef enum {
    DISPONIVEL,
    CARREGANDO,
    CONCLUIDO
} StatusSessao;

typedef struct {
    int id; // ID da sessão
    char modeloVeiculo[30]; // Modelo do carro
    float energiaConsumida; // Energia em kWh
    float tempoRecarga; // Tempo em minutos
    float custoTotal; // Custo em R$
    StatusSessao status; // Status da sessão
} Sessao;
```

Para guardar os registros, o programa usa um vetor estático de estruturas: `Sessao sessoes[100]`, que suporta até 100 recargas. O controle da quantidade de posições ocupadas no vetor é feito pela variável `totalSessoes`, o que impede acessos indevidos fora dos limites da memória.

## 2. FUNCIONAMENTO DO PROGRAMA

---

O sistema funciona através de um menu interativo no terminal e foi estruturado totalmente em funções para manter o código organizado:

- **exibirMenu():** Mostra as opções do menu (1 a 6) dentro de um laço `do-while` que só encerra quando o usuário digita a opção 6. Inclui tratamento do buffer de entrada para evitar travamentos.
- **cadastrarSessao():** Pede os dados do veículo, energia consumida e tempo com validação de entrada nos `scanf`. Calcula o custo total com base na tarifa padrão, atribui um ID automático e salva a sessão no vetor.

- **listarSessoes():** Percorre o vetor exibindo todas as sessões cadastradas até o momento em formato de tabela, utilizando a função `obterStatus()` para ler dinamicamente o status armazenado na `struct`.
- **buscarSessao():** Permite pesquisar uma sessão específica informando o ID. O usuário pode escolher utilizar a Busca Linear ou a Busca Binária.
- **ordenarSessoes():** Permite reordenar o vetor por ID, energia consumida, custo ou tempo de recarga.
- **mostrarEstatisticas():** Calcula e exibe o total de sessões, a energia total distribuída, o faturamento total, o ticket médio por sessão e os valores de maior e menor consumo registrados.

### 3. ALGORITMOS DE BUSCA UTILIZADOS

---

Foram implementados manualmente dois algoritmos de busca:

#### 3.1. Busca Linear

A função `buscaLinearPorID()` percorre o vetor posição por posição (da primeira até a última) comparando o ID informado com o ID salvo em cada posição. Funciona normalmente mesmo que o vetor esteja desordenado.

#### 3.2. Busca Binária

A função `buscaBinariaPorID()` divide o vetor ao meio a cada passo para encontrar o elemento desejado. Como a busca binária exige obrigatoriamente que o vetor esteja ordenado pelo campo buscado, a função executa a ordenação por ID (`insertionSort`) antes de realizar a pesquisa.

### 4. ALGORITMO DE ORDENAÇÃO UTILIZADO

---

Foi implementado manualmente o algoritmo **Insertion Sort** na função `insertionSort()`. Ele funciona inserindo cada elemento do vetor em sua posição correta na parte já ordenada, deslocando os elementos maiores para a direita.

O algoritmo foi feito do zero (sem usar bibliotecas como a `qsort`) e permite escolher por qual campo o vetor será ordenado:

1. ID
2. Energia Consumida (kWh)
3. Custo Total (R\$)
4. Tempo de Recarga (min)

## 5. ANÁLISE BIG-O DE COMPLEXIDADE

---

### 5.1. Insertion Sort — $O(n^2)$

**Melhor caso —  $O(n)$ :** Ocorre quando o vetor já está totalmente ordenado. O laço externo roda  $n - 1$  vezes e o laço interno não realiza trocas.

**Pior caso —  $O(n^2)$ :** Ocorre quando o vetor está totalmente invertido em relação à ordem desejada.

**Análise pelo código:**

```
void insertionSort(Sessao sessoes[], int total, int criterio) {
    for (int i = 1; i < total; i++) {    // Laço externo: (n - 1) vezes
        Sessao chave = sessoes[i];
        int j = i - 1;

        while (j >= 0 && condicao) {      // Laço interno: até 'i' vezes
            sessoes[j + 1] = sessoes[j]; // Deslocamento de memória
            j--;
        }
        sessoes[j + 1] = chave;
    }
}
```

Como o laço externo executa  $n - 1$  vezes e o laço interno faz até  $i$  comparações e trocas em cada iteração no pior caso, o número total de operações é a soma:  $1 + 2 + 3 + \dots + (n - 1) = [n * (n - 1)] / 2 = (n^2 - n) / 2$ . Desprezando os termos menores e constantes, a complexidade no pior caso é  $O(n^2)$ .

### 5.2. Busca Binária — $O(\log n)$ e Análise do Encadeamento no Programa

**Melhor caso do algoritmo de busca —  $O(1)$ :** Ocorre quando o elemento procurado está exatamente no meio do vetor na primeira verificação.

**Pior caso do algoritmo de busca —  $O(\log n)$ :** Ocorre quando o elemento procurado está em uma das extremidades ou não está no vetor.

**Análise pelo código:**

```
int buscaBinariaPorID(Sessao sessoes[], int total, int idBusca) {
    insertionSort(sessoes, total, 1);    // Ordenação previa:  $O(n^2)$ 

    int inicio = 0, fim = total - 1;
    while (inicio <= fim) {              // Divide a busca ao meio:  $O(\log n)$ 
        int meio = inicio + (fim - inicio) / 2;
        if (sessoes[meio].id == idBusca) return meio;

        if (sessoes[meio].id < idBusca)
            inicio = meio + 1;
        else
            fim = meio - 1;
    }
    return -1;
}
```

**Nota de Análise de Complexidade Composta:** O algoritmo de Busca Binária isoladamente possui complexidade estrita  $O(\log n)$  no pior caso, pois a cada iteração do laço `while` o tamanho do vetor é cortado pela metade ( $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$ ), resultando em  $\log_2(n)$  divisões.

No entanto, considerando a função do sistema `buscaBinariaPorID()` de forma integrada, é necessário garantir a ordenação do vetor antes de executar a divisão e conquista. Como a ordenação utiliza o `insertionSort` (cuja complexidade no pior caso é  $O(n^2)$ ), a complexidade total da operação encadeada torna-se  $O(n^2) + O(\log n) = O(n^2)$ .